



Instituto Superior de Ciências e Tecnologia de Moçambique

Engenharia Informática

Inteligência Artificial

TurRoute

Planeamento de Circuitos Turísticos em Moçambique

Grupo 1, Tema 4.2.8:

20240143 — Aashir Omar

20240459 — Igor dos Santos

20240567 — Stanley Cossa

20240526 — Quirson Ngale

20220057 — Saudah Ahmad

20230414 — Hélder Júnior

Docente:

Mestre Ruben Manhiça

Maputo, 28 de Maio de 2026

Índice

Índice	2
1. Introdução	3
2. Descrição do Problema	3
2.1 Objetivos do Agente	3
2.2 Requisitos Funcionais e da Interface	4
2.3 Requisitos	5
2.4 Saídas produzidas pelo sistema	5
3. Modelação do Problema	6
3.1 Representação	6
3.2 Dados geográficos	7
3.3 Cálculo de distâncias (Fórmula de Haversine)	9
4. Algoritmos de Busca	9
4.1 Greedy: Vizinho Mais Próximo	9
4.2 BFS: Busca em Largura	10
4.3 DFS: Busca em Profundidade	11
4.4 A* com Heurística MST: Busca Informada	11
4.5 Comparação dos algoritmos	13
5. Resultados Experimentais	13
5.1 Cenário 1: Circuito Sul (4 cidades)	13
5.2 Cenário 2: Circuito Completo (11 cidades)	14
5.3 Análise dos resultados (com base no cenário 2)	14
6. Fluxo de Execução	15
7. Interface e Visualização	15
8. Testes e Validação	16
10. Conclusão	17
11. Referências	18

1. Introdução

O TurRoute é um sistema inteligente de optimização de circuitos turísticos em Moçambique, desenvolvido com o objectivo de resolver o Problema do Caixeiro Viajante (TSP ou *Travelling Salesman Problem*) aplicado à rede de voos domésticos da LAM (Linhas Aéreas de Moçambique). O problema TSP consiste em determinar a ordem óptima de visita a um conjunto de cidades e retorno a origem, minimizando a distância total percorrida. Apesar de simples na enunciação, o TSP pertence à classe de problemas NP-difícil: com 11 cidades, existem 3 628 800 rotas possíveis, tornando a verificação exaustiva computacionalmente inviável. Logo, a solução implementada combina quatro algoritmos de busca i.e. Greedy, BFS, DFS e A* com heurística MST (árvore mínima de cobertura), numa aplicação web completa com mapa interactivo, animação de rotas e comparação visual de desempenho.

2. Descrição do Problema

2.1 Objetivos do Agente

O enunciado propõe o desenvolvimento de um sistema para planear uma viagem entre várias cidades, inspirado no Problema do Caixeiro Viajante (TSP). O agente deve:

- Partir de uma cidade inicial definida pelo utilizador;
- Visitar todas as cidades seleccionadas exactamente uma vez;
- Minimizar o custo total da viagem (i.e. distância total percorrida).

2.2 Requisitos Funcionais e da Interface

Requisitos Funcionais	
Requisito	Implementação
Definir número de cidades	Toggles na <i>sidebar</i> i.e. selecção de 2 a 11 cidades
Geração automática da Matriz de distâncias	Matriz gerada automaticamente pela fórmula de Haversine a partir de coordenadas reais.
Escolher cidade inicial	Secção <i>dropdown</i> na interface “Cidade de Partida”
Executar algoritmos de busca	4 algoritmos i.e. Busca Gulosa, BFS, DFS, A*
Requisitos da Interface	
Requisito	Implementação
Mostrar cidades num plano (2D)	Imagem do Mapa com posicionamento geográfico real de Moçambique
Representar ligações entre cidades	Arcos no mapa; <i>badge</i> “escala” para segmentos sem voo directo LAM
Visualizar o percurso final	Animação em tempo real do avião ao longo da rota calculada
Destacar a ordem de visita	Sequência numerada e ordenada na rota apresentada

2.3 Requisitos

1. Implementação da heurística admissível para A*: admissibilidade garantida e implementada via Minimum Spanning Tree (*MST* ou árvore mínima de cobertura) com o Algoritmo de *Prim*.
2. Comparação ótimo vs aproximado: A* (ótimo garantido) comparado directamente com busca gulosa (heurística construtiva).
3. Suporte inteligente a número elevado de cidades (optimização necessária).

2.4 Saídas produzidas pelo sistema

Para cada algoritmo executado, o sistema devolve:

Saída esperada	Onde é apresentada / Como é implementada
Ordem de visita das cidades	Rota apresentada: Maputo → Beira → ... etc.
Custo (Distância) total em <i>km</i>	Métrica principal (no painel de resultados)
Nós explorados	Coluna na tabela de comparação + gráfico de barras
Tempo de execução	
Tempo Estimado de Voo do circuito completo	Usa a velocidade de avião da LAM @ 500 km/h e identifica automaticamente os segmentos sem voo directo na rede LAM. O tempo de voo inclui o regresso à cidade de partida.

3. Modelação do Problema

3.1 Representação

	Implementação
Estado	Par (cidade_actual, frozenset(cidades_visitadas))
Estado inicial	(cidade_partida, {cidade_partida})
Acções possíveis	Voar para qualquer cidade ainda não visitada
Teste de objectivo	$\text{len}(\text{cidades_visitadas}) == \text{total_cidades_seleccionadas}$
Custo de passo	Distância geodésica (Haversine) entre as duas cidades
Custo total	Soma acumulada de todas as distâncias percorridas

O uso de frozenset para representar o conjunto de cidades visitadas é essencial, permitindo que o mesmo conjunto de cidades visitadas seja reconhecido como estado equivalente independentemente da ordem em que foram visitadas, evitando redundância na exploração.

3.2 Dados geográficos

A rede de voos é baseada nas rotas reais da LAM.

As 11 cidades cobertas pelo sistema são:

Cidade	Região	Latitude	Longitude
Maputo	Sul	-25.9667	32.5833
Inhambane	Sul	-23.8650	35.3833
Vilanculos	Sul	-21.9833	35.3167
Beira	Centro	-19.8436	34.8389
Chimoio	Centro	-19.1167	33.4833
Quelimane	Centro	-17.8786	36.8881
Tete	Centro	-16.1564	33.5867
Nampula	Norte	-15.1167	39.2667
Nacala	Norte	-14.5422	40.6783
Lichinga	Norte	-13.3147	35.2392
Pemba	Norte	-12.9667	40.5167

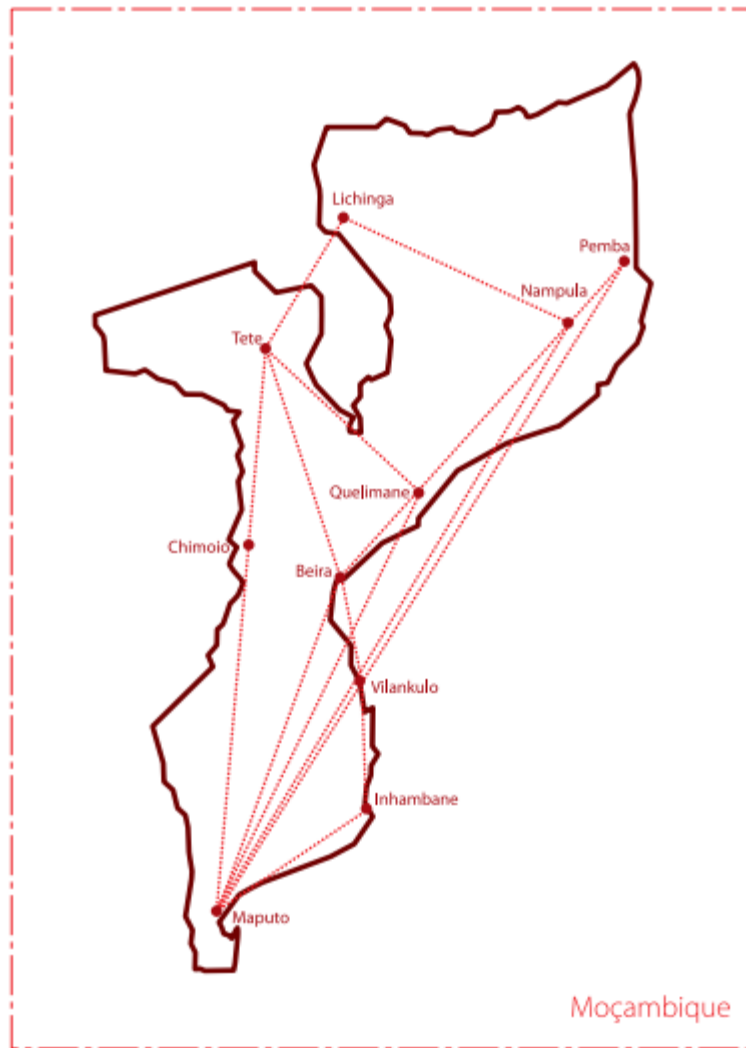


Figura 1 — Rede de rotas domésticas da LAM (fonte: lam.co.mz)

3.3 Cálculo de distâncias (Fórmula de Haversine)

As distâncias entre cidades são calculadas pela fórmula de *Haversine* (usada amplamente na navegação e aeronáutica), que determina a distância real entre dois pontos na superfície terrestre a partir das suas coordenadas geográficas:

$$a = \sin^2(\Delta lat/2) + \cos(lat_1) \cdot \cos(lat_2) \cdot \sin^2(\Delta lon/2)$$

$$c = 2 \cdot \operatorname{atan2}(\sqrt{a}, \sqrt{1-a})$$

$$d = R \cdot c$$

onde $R = 6371$ km i.e. raio médio da Terra

$lat = latitude$ e $long = longitude$.

A fórmula garante precisão geográfica real, ao contrário da distância euclidiana que seria inadequada para coordenadas esféricas.

4. Algoritmos de Busca

4.1 Greedy: Vizinho Mais Próximo

- **Estratégia:** a cada passo, selecciona a cidade não visitada mais próxima da posição actual.
- **Tempo:** $O(N^2)$
- **Espaço:** $O(N)$
- **Optimalidade:** Não. A busca gulosa é uma heurística construtiva. Pode chegar a soluções de 20 a 30% piores que o óptimo em casos desfavoráveis, porque decisões localmente óptimas (o próximo passo só depende da menor heurística actual) podem comprometer o percurso global.
- **Uso recomendado:** circuitos com muitas cidades onde velocidade de resposta é priorizada sobre a optimalidade.

- **Implementação:**

```
def greedy(problem):  
    current = problem.initial_state  
    visited = {current.city}  
    path = [current.city]  
    total_cost = 0  
  
    while len(visited) < len(problem.cities):  
        nearest = min(  
            (c for c in problem.cities if c not in visited),  
            key=lambda c: problem.distance(current.city, c)  
        )  
        total_cost += problem.distance(current.city, nearest)  
        current = State(nearest, frozenset(visited | {nearest}))  
        visited.add(nearest)  
        path.append(nearest)  
  
    return path, total_cost
```

4.2 BFS: Busca em Largura

- **Estratégia:** explora todos os estados ao mesmo nível de profundidade antes de avançar para o seguinte. Usa uma fila *FIFO* (o primeiro elemento a entrar é o primeiro a sair).
- **Tempo & Espaço:** $O(N!)$ i.e. cresce factorialmente com o número de cidades.
- **Optimalidade:** ótimo, em número de passos. Com distâncias variáveis, o BFS não garante a rota de menor custo - apenas a de menor número de voos. Para custo ótimo, o A* é o algoritmo correcto.
- **Limitação implementada:** o programa bloqueia o BFS para circuitos com mais de 10 cidades, substituindo automaticamente pela Busca Gulosa com aviso ao utilizador, evitando o acto de explorar potencialmente milhões de estados, tornando-o impraticável.

4.3 DFS: Busca em Profundidade

- **Estratégia:** explora um caminho até ao fim (beco sem saída) antes de recuar e tentar outro. Usa uma pilha *LIFO* (o último a entrar é o primeiro a sair).
- **Tempo:** $O(N!)$
- **Espaço:** $O(N)$, logo mais eficiente em memória que o BFS, que usa $O(N!)$.
- **Optimalidade:** Não, pois explora sem critério de qualidade, podendo percorrer longos ramos de custo elevado antes de encontrar soluções melhores.
- **Nota de implementação:** os resultados do DFS podem variar entre execuções devido ao comportamento de iteração sobre frozenset em *Python*, que não garante ordem determinista.

Resultado com 11 cidades: 23 051 estados explorados, rota de 5 524 km i.e 87% pior que o óptimo.

4.4 A* com Heurística MST: Busca Informada

Algoritmo principal do sistema que garante a rota óptima de forma eficiente.

- **Estratégia:** expande sempre o estado na fronteira com menor valor usando o valor f

$$f = g + h$$

- g = custo real acumulado (quilómetros já voados)
- h = estimativa do custo mínimo restante

Usa uma fila de prioridade (heap mínimo) para seleccionar eficientemente o estado mais desejável

- **A* Heurística: árvore mínima de cobertura (MST):**

Para estimar o custo restante, o A* calcula a Árvore de Cobertura Mínima (MST) das cidades ainda não visitadas, usando o Algoritmo de Prim:

```
def mst_heuristic(unvisited_cities, distances):
    if len(unvisited_cities) <= 1:
        return 0

    cities = list(unvisited_cities)
    in_tree = {cities[0]}
    min_cost = 0

    while len(in_tree) < len(cities):
        edge = min(
            (distances[u][v] for u in in_tree for v in cities if
v not in in_tree),
            default=0
        )
        min_cost += edge
        for u in in_tree:
            for v in cities:
                if v not in in_tree and distances[u][v] == edge:
                    in_tree.add(v)
                    break

    return min_cost
```

- **Por que é admissível:** a MST representa o custo mínimo absoluto de conectar todas as cidades restantes. Qualquer rota real que as visite tem de as conectar de alguma forma, e nunca pode fazê-lo com um custo inferior à MST. Logo, h nunca sobrestima o custo real (heurística admissível)
- **Teorema:** uma heurística admissível garante que o A* encontra sempre a solução ótima.
- **Resultado com 11 cidades:** apenas 21 estados explorados, rota ótima de 2951 km.

4.5 Comparação dos algoritmos

Critério	Greedy	BFS	DFS	A* (MST)
Optimalidade	Não	Sim (passos)	Não	Sim (custo)
Tempo	$O(N^2)$	$O(N!)$	$O(N!)$	$O(N^2 \cdot 2^N)$
Espaço	$O(N)$	$O(N!)$	$O(N)$	$O(N \cdot 2^N)$
Estrutura de dados	Lista	Fila FIFO	Pilha LIFO	Fila de Prioridade
Limite prático	Sem limite	≤ 10 cidades	500 000 nós	≤ 12 cidades

5. Resultados Experimentais

5.1 Cenário 1: Circuito Sul (4 cidades)

Cidades: Maputo, Inhambane, Vilanculos, Beira.

Partida: Maputo

Algoritmo	Rota	Custo (km)	Tempo de voo (HH:MM)	Qtd. Nós explorados
A*	Maputo → Inhambane → Vilanculos → Beira	819.8	03:05	4
Greedy	Maputo → Inhambane → Vilanculos → Beira	819.8	03:05	4
BFS	Maputo → Inhambane → Vilanculos → Beira	819.8	03:05	16
DFS	Maputo → Inhambane → Beira → Vilanculos	1063.0	03:10	16

O Greedy encontrou o óptimo por coincidência (casualmente). Não é comportamento garantido.

5.2 Cenário 2: Circuito Completo (11 cidades)

Cidades: Todas as cidades da rede LAM

Partida: Maputo

Algoritmo	Custo (km)	Tempo de voo (HH:MM)	Qtd. Nós explorados
A* (MST)	2951	09:14	21
Greedy	3165	08:20	11
BFS	Bloqueado: mais que 10 cidades	Bloqueado: mais que 10 cidades	0
DFS	5524	14:06	23 051

5.3 Análise dos resultados (com base no cenário 2)

- O A* explorou 1098 vezes menos estados que o DFS e encontrou uma rota 47% mais curta.
 - $23\ 051 \div 21 = 1098$ vezes
 - $((5524-2951) \div 5524) * 100 = 47\%$
- A busca gulosa oferece um compromisso razoável i.e. rota 7% pior que o óptimo, mas com resposta quase instantânea.
- O DFS confirmou o comportamento esperado (de acordo com a complexidade): rápido em memória, mas sem critério de qualidade i.e. produz soluções muito distantes do óptimo.
- O BFS é ineficiente para circuitos grandes, mas garante optimalidade em grafos pequenos.

6. Fluxo de Execução

1. Utilizador selecciona cidades e algoritmo

↓

2. Interface valida input (mínimo 2 cidades, partida válida)

↓

3. Servidor corre algoritmo(s) seleccionado(s)

4. Calcula tempo de voo e identifica segmentos sem voo directo

↓

5. Interface desenha rota no mapa

6. Anima o avião ao longo do trajecto

7. Apresenta gráficos comparativos

7. Interface e Visualização

A interface foi desenvolvida como uma SPA (*Single Page Application*) em HTML, CSS e JavaScript puro, sem dependências de framework front-end.

Funcionalidades principais da interface:

- Mapa SVG de Moçambique com posicionamento geográfico real das cidades
- Animação do avião a percorrer a rota calculada
- Gráficos de comparação entre algoritmos (*Chart.js*): custo, nós explorados, tempo de execução
- Identificação automática de escalas: quando um segmento não tem voo directo na rede LAM
- Três presets de cenário: Sul, Centro-Norte, Completo
- Adaptação automática da interface conforme o algoritmo seleccionado (um ou todos)

8. Testes e Validação

O sistema inclui 48 testes automatizados organizados em três ficheiros:

Ficheiro	Qtd de testes	Âmbito
<i>test_integration.py</i>	14	Testes end-to-end da API e integração
<i>test_edge_cases.py</i>	17	Casos extremos: 2 cidades, cidades inválidas, BFS com 11+ cidades
<i>test_metrics.py</i>	15	Validação de métricas: distâncias, tempo de voo, formatação

Todos os 48 testes passam. Para executar:

```
python -m pytest tests/ -v
```


10. Conclusão

O TurRoute demonstra que a escolha do algoritmo de busca tem impacto directo e significativo na qualidade da solução encontrada. Para o problema TSP aplicado à rede LAM, A^* é a solução correcta quando se pretende o óptimo garantido: explorou apenas 21 estados para encontrar a rota de 2 951 km com 11 cidades. Por outro lado, o Greedy é uma alternativa prática e rápida para cenários onde velocidade importa mais que perfeição e a optimalidade. O DFS confirmou as suas limitações teóricas i.e. não possui critério de qualidade e produz soluções muito afastadas do óptimo enquanto o BFS é adequado apenas para instâncias pequenas, dada a sua complexidade factorial. A admissibilidade da heurística MST, demonstrada pelo facto de a MST representar o custo mínimo absoluto de conectar as cidades restantes, é a propriedade integral que garante a optimalidade do A^* . Esta relação entre teoria e resultados empíricos é o contributo principal deste trabalho.

11. Referências

- Russell, S. & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4ª ed.). Pearson.
- Cormen, T. H., et al. (2009). Introduction to Algorithms (3ª ed.). MIT Press.
- Documentação oficial Flask: <https://flask.palletsprojects.com>
- Documentação Chart.js: <https://www.chartjs.org>
- LAM — Linhas Aéreas de Moçambique, rede de rotas domésticas: <https://www.lam.co.mz>